DEPLOYGUARD

Multi-Component Deployment

Blockers, Root Causes, Architecture Corrections & Final Live Proof

Purpose

A Viva-focused engineering report documenting the problems encountered while moving DeployGuard from a single-root deployment model to a bounded multi-component application model, and the exact solutions that made the final React/Vite + Express + managed MongoDB deployment reach LIVE.

Final controlled outcome

LIVE - frontend + backend + managed MongoDB, verified and promoted

Evidence	Value
Production endpoint	p-87a44322-ef32-47f2-a0b1-f9682da91937.apps.deployguard.me
Final operation	b00a0b4f-083e-47fe-8ef3-f83511dfb283
Generation	b0840bb1-69f2-4386-921c-d14ba4e7efd1
Candidate / promotion	31943120757 / 31943574176
Persistent EFS	fs-08ad971fdfea4ba8c

Prepared for FYP / Viva Revision | 16 August 2026

1. Executive Summary

The multi-component testing phase exposed a chain of platform-level correctness defects that did not appear in simple single-component deployments. The most important lesson was that a repository cannot be treated as one deployable root when it actually represents a frontend, backend, and optional managed database.

Core architectural shift

Before: repository -> selected root -> one BuildPlan -> one image -> one container -> LIVE. After: repository -> bounded topology -> componentized BuildPlan -> per-component images -> sibling containers in one ECS task -> complete-component verification -> LIVE.

Problems encountered in sequence

#	Blocker	What it broke	Resolution
1	Single-root topology collapse	Frontend could be silently omitted while backend became LIVE	Componentized topology + fail-closed ambiguity
2	Frontend ENV misclassification	Optional public Vite key treated as private secret	Global ownership/exposure/phase/requiredness model
3	Routing prefix defect	Every /api/* request lost the /api prefix and returned 404	Deterministic preserve/strip relationship semantics
4	Persistence admission mismatch	Existing EFS was reported unavailable	Workflow-compatible deterministic EFS identity
5	Optional ENV became mandatory	Dispatch stopped although readiness allowed omission	Materialization now respects requiredInputs
6	OIDC IAM gap	Authoritative candidate verification could not DescribeTasks	Added only ecs:DescribeTasks

Final result

- The public frontend is reachable and healthy.
- The backend is reachable through the declared frontend relationship without prefix loss.
- The managed MongoDB service and EFS persistence remain project-scoped and healthy.
- Candidate verification proves exact ECS task definition, image identities, container health, ALB health, frontend availability, and frontend-to-backend routing.
- Promotion succeeded and the final generation reached LIVE.

2. Architecture Evolution

2.1 Original architecture

```
repository
  |
  v
candidate discovery
  |
  v
select ONE applicationRoot
  |
  v
one BuildPlan
  |
  v
one Docker image / one runtime
  |
  v
single health result
  |
  v
LIVE
```

This model was adequate for frontend-only or backend-only repositories, but it was fundamentally unable to represent an intentional frontend + backend topology.

2.2 Corrected bounded multi-component architecture

```
repository
  |
  v
bounded topology discovery
  |
  +--> frontend component
  +--> optional backend component
  +--> optional backend-owned managed database
  |
  v
componentized BuildPlan + relationships[]
  |
  +--> independent frontend build context/image
  +--> independent backend build context/image
  |
  v
ONE immutable ECS release / sibling containers
  |
  +--> frontend :8080  (public through ALB)
  +--> backend  :3000/5000 (task-local)
  |
  v
frontend + backend + relationship + DB verification
  |
```

```
  v
LIVE
```

2.3 Supported bounded scope

- One frontend component.
- Optional one backend component.
- Optional managed database owned only by the backend.
- Existing single-component deployments remain backward compatible.
- Ambiguous topologies such as one frontend plus two backends fail closed instead of silently choosing one.

> **Why this matters in the Viva**
>
> The correction was not 'support two Dockerfiles'. It changed the deployment contract from a singular application-root assumption into an explicit component topology with roles, relationships, per-component build contexts, and complete-release verification.

3. Blocker 1 - Multi-Component Repository Collapsed to One Root

Problem

Repository analysis could discover multiple application candidates but persisted exactly one highest-scoring applicationRoot. Every downstream layer - BuildPlan, Docker, GitHub Actions, Terraform, health verification, and LIVE promotion - therefore represented only one component.

Observed failure

```
Library Management System
├── Frontend/   (plain HTML/CSS/JS)
└── Backend/    (Express + MongoDB)

Detector result:
applicationRoot = Backend

Deployment result:
Backend deployed and healthy
Frontend absent
Project could still become LIVE  <-- false positive
```

Root cause

- BuildPlan was singular: one appRoot, framework, Docker strategy, port, and health path.
- Static frontends without package/build manifests were invisible to candidate discovery.
- Docker correctly built only the selected root because GitHub Actions changed into APPLICATION_ROOT before docker build.
- LIVE verification proved one service, not the complete repository surface.

Solution implemented

- Added bounded repository topology discovery, including static frontend roots.

- Extended BuildPlan with components[] and relationships[] while deriving legacy singular fields for backward compatibility.
- Created independent Docker build contexts and immutable images per component.
- Represented frontend/backend routing explicitly.
- Changed readiness to block unsupported or ambiguous topologies instead of guessing.
- Changed LIVE semantics to require the complete declared component inventory and relationships.

Additional defects exposed during this architecture change

Defect	Correction
Workflow component validation used an incorrect jq all() expression	Fixed component-aware workflow validation.
Static nginx container received a Node.js health command	Health behavior became component/runtime-specific.
LIVE finalization checked runtime identity against the static frontend	Runtime-bearing backend identity is now verified correctly.

4. Blocker 2 - Environment Configuration Was Too Strict

Observed failure

The external full-stack repository was correctly detected as Vite/React frontend + Express backend + backend-owned MongoDB, but readiness blocked on VITE_WEATHER_API_KEY with 'Secret variables cannot be used during image build.'

Exact root cause

Secret-name heuristics were applied before framework exposure semantics. A public frontend build variable containing API_KEY in its name was treated as a private secret. Requiredness was also inferred too broadly.

Global solution - not a React-only patch

Dimension	Meaning
Owner/component	frontend, backend, platform, or managed database
Exposure	public/client-visible or private
Phase	build-time or runtime
Requirement	required, optional, or unknown

- Framework-public prefixes such as VITE_*, REACT_APP_*, and NEXT_PUBLIC_* are treated as client-visible configuration, not private solely because their names contain KEY/TOKEN/SECRET.
- Backend secrets such as JWT_SECRET remain private runtime configuration.
- PORT and infrastructure values remain DeployGuard-owned.
- Managed database connection values remain backend-only and DeployGuard-owned.

- Optional public configuration may be absent with a warning; required missing configuration blocks; unknown dangerous configuration fails closed.

Result

For Fullstack-Web-Application: JWT_SECRET remained a private backend runtime value; VITE_WEATHER_API_KEY became optional public frontend build configuration; MongoDB and ports remained platform-owned. Readiness advanced to READY_WITH_WARNINGS.

5. Blocker 3 - /api Prefix Was Stripped from Every Backend Request

Observed runtime failure

```
Expected:
GET /api/health -> backend /api/health -> 200

Actual:
GET /api/health
  -> nginx stripped /api
  -> backend /health
  -> 404

The same happened to all routes:
GET /api/tasks -> backend /tasks -> 404
```

Exact root cause

stripPathPrefix was guessed from the backend health path. Because the backend health path /healthz did not contain /api, the generated nginx proxy used trailing-slash semantics that stripped /api from every proxied request. Static frontend health also incorrectly inherited /api/health.

Solution implemented

- Frontend and backend route declarations are matched deterministically.
- If both sides own /api/*, the prefix is preserved.
- If the public side exposes /api/* but the backend is rooted at /*, the prefix is stripped.
- Ambiguous route relationships block readiness rather than guessing.
- A persisted verificationPath records the public route that must be proven during candidate verification.

Health responsibilities were separated

Verification	Rule
Frontend / ALB	GET / proves nginx and static frontend independently.
Backend runtime	Backend component keeps its own health path and ECS container HEALTHY evidence.
Frontend -> backend	Workflow requests the persisted relationship verificationPath.

Live proof after correction

Endpoint	Result	Meaning
/	200	Frontend reachable.
/api/health	200	Public relationship reaches the backend with prefix preserved.
/api/tasks	401 Authentication required	Request reached Express auth middleware; 401 is expected without credentials.

6. Blocker 4 - Existing Persistent MongoDB Storage Was Reported Unavailable

Observed failure

After the routing correction, a controlled redeploy stopped before GitHub dispatch. DeployGuard reported that previous persistent data was unavailable even though the managed MongoDB service remained ACTIVE 1/1.

Exact root cause

Backend EFS reconciliation and the reusable workflow generated different deterministic creation tokens.

> Workflow:
> dg-efs-<project-prefix>-<40-char-hash>
>
> Backend:
> dg-efs-<project-prefix>-<environment>-<32-char-hash>
>
> Existing EFS:
> fs-08ad971fdfea4ba8c
>
> Actual creation token:
> dg-efs-87a44322-cc1dc6c6028be0e3f948b24a30ead084504828a4

Why fail-closed was correct

DeployGuard was right to refuse a redeploy when it could not prove the persistent filesystem identity. The defect was the lookup identity supplied to that safety check, not the safety check itself.

Solution implemented

- Prefer an exact bound EFS ID when one is available.
- Otherwise derive the exact workflow-compatible deterministic creation token.
- Require ownership tags: ManagedBy, project ID, environment, and Persistence=project.
- Missing, ambiguous, or ownership-mismatched persistence remains blocked.
- No automatic database recreation or cross-project adoption is allowed.

Result

Reconciliation found the existing EFS and access point, returned HEALTHY / UPDATE / PERSISTENT, and allowed deployment while preserving the existing MongoDB data path.

7. Blocker 5 - Optional ENV Became Mandatory at Dispatch

Observed failure

Persistence admission passed, but dispatch stopped because VITE_WEATHER_API_KEY had no persisted value - even though readiness had already classified it as optional and allowed its absence.

Exact root cause

buildTimePublicConfig() iterated all buildTimeEnvVars and treated every missing persisted row as required. It ignored plan.requiredInputs.

Why this was a contract inconsistency

```
Readiness:
VITE_WEATHER_API_KEY = optional + absent
=> READY_WITH_WARNINGS

Dispatch:
missing persisted value
=> BLOCKED

Correct invariant:
readiness and dispatch must consume the same authoritative requiredness.
```

Solution implemented

Configuration state	Materialization behavior
Required + missing	Block before dispatch.
Optional + missing	Omit the variable and continue.
Optional public build value + configured	Decrypt and include only in the relevant frontend build.
Private/secret value in public build	Reject.
Platform/database-owned	Resolve internally; do not demand user values.

Controlled result

- VITE_WEATHER_API_KEY was omitted as intended.
- GitHub dispatch succeeded.
- Frontend and backend images built.
- Persistence preparation and Terraform passed.
- The corrected multi-component candidate was launched successfully.

8. Blocker 6 - GitHub OIDC Role Could Not Authoritatively Verify ECS Tasks

Observed failure

The corrected candidate was actually running: frontend returned 200, /api/health returned 200, /api/tasks reached the backend, the ALB target was healthy, and both ECS application and MongoDB services were ACTIVE 1/1. Promotion still stopped.

Exact root cause

Candidate verification calls ecs:ListTasks and then ecs:DescribeTasks to prove the running backend container's health and immutable image identity. The GitHub OIDC role lacked ecs:DescribeTasks.

Why DeployGuard was right not to promote

HTTP success alone is not authoritative release identity. DeployGuard must prove that the expected task definition and exact component images are running. Failing closed when AWS identity verification is denied preserves the meaning of LIVE.

Least-privilege correction

Item	Final state
Role	DeployGuardGitHubActionsProjectTst
Inline policy	DeployGuardWorkflowCapabilities
Permission added	ecs:DescribeTasks only
Already present	ecs:ListTasks, ecs:DescribeServices, ecs:DescribeTaskDefinition
Broad wildcard	None - no ecs:* and no administrator access

Controlled rerun

Evidence	Result
Attempt	5
Operation	b00a0b4f-083e-47fe-8ef3-f83511dfb283
Candidate run	31943120757 - succeeded
Promotion run	31943574176 - succeeded
Final state	LIVE

9. Final End-to-End Proof

9.1 Final application topology

```
Internet
  |
  v
Application Load Balancer
  |
  v
Frontend container (nginx, :8080)
  |    |  \-- serves frontend assets
  |
  +-- /api/* --> task-local backend container
                    |
                    v
              Express backend
                    |
                    v
          DeployGuard-managed MongoDB
                    |
                    v
                  EFS
```

9.2 What LIVE now proves

- Exact declared component inventory.
- Exact frontend and backend image digests.
- Exact ECS task definition and service.
- ECS stability and component/container health.
- Healthy ALB target and reachable frontend.
- Reachable backend through the declared frontend relationship.
- Managed database readiness when declared.
- Candidate-route removal and stable-route promotion.
- Successful authoritative stable-release persistence.

9.3 Final endpoint evidence

Check	Final result
Frontend /	HTTP 200
Backend relationship /api/health	HTTP 200
Protected API /api/tasks	HTTP 401 Authentication required - correct backend reachability
ECS application	ACTIVE 1/1
Managed MongoDB	ACTIVE 1/1
Promotion	Succeeded
Release state	LIVE

Final conclusion

The public repository became a strong black-box certification case because DeployGuard was not modified merely to 'make the app pass'. Each failure was traced to the owning layer, fixed as a bounded platform invariant, regression-tested, and then retested through the controlled deployment path.

10. Viva Cheat Sheet - Problem -> Root Cause -> Solution

Problem	Root cause	Implemented solution
Only backend deployed	Singular applicationRoot / BuildPlan	components[] + relationships[] + static frontend discovery + fail closed
Optional Vite key blocked readiness	Name-only secret heuristics	Global owner/exposure/phase/requirement ENV model
/api routes returned 404	Prefix stripping guessed from health path	Evidence-based preserve/strip routing + verificationPath
Existing EFS 'unavailable'	Backend/workflow token formulas differed	Canonical deterministic EFS identity + ownership tags
Optional ENV blocked dispatch	Materializer ignored requiredInputs	Missing optional values are omitted
Candidate could not promote	OIDC role lacked ecs:DescribeTasks	Added exact read permission; kept least privilege

30-second explanation

While testing a real multi-component repository, we discovered that our original deployment contract assumed one application root. That could silently drop a frontend and still mark the backend LIVE. We replaced the singular model with a bounded component topology, independent component images, explicit frontend-backend relationships, backend-owned database binding, and complete-release verification. The live test then exposed narrower defects in ENV ownership, route-prefix semantics, persistence identity, optional ENV materialization, and IAM verification. Each was fixed at its owning layer, and the final React/Vite + Express + MongoDB generation passed candidate verification, promotion, and reached LIVE.

Key design principles to remember

- Fail closed when DeployGuard cannot prove topology, persistence ownership, or release identity.
- Do not solve platform defects by modifying the test application just to make it pass.
- Application configuration, platform configuration, and database configuration have different owners.
- A 200 response is not enough for LIVE; exact component identity and relationships must also be proven.
- Prefer bounded support with deterministic behavior over broad 'universal' guessing.
- Preserve backward compatibility for valid single-component repositories.